Rapport : Mise en place d’une architecture DMZ via GNS3 et Docker

Membres du groupe :

- 1979
- 2075

Table des matieres :

- Rapport : Mise en place d’une architecture DMZ via GNS3 et Docker
  - Membres du groupe :
  - Table des matieres :
  - Introduction
  - Méthodologie
    - **Environnement**
    - 1. **Création des images Docker**
      - 1.1 Client LAN (`ubuntu_client`)
      - 1.2 Pare-feu (`ubuntu_firewall`)
      - 1.3 Serveur Web DMZ (`ubuntu_webserver`)
      - 1.4 Serveur Web LAN (`ubuntu_lan_webserver`)
      - 1.5 WAN (`ubuntu_wan`)
    - 2. **Mise en place de la topologie**
      - 2.1 **Création des containers dockers dans gns3**
        - firewall
        - public-server
        - internal-server
        - internal-desktop
        - internet
      - 2.2 **Mise en place de la topologie dans gns3**
        - 1. Instanciation et liaison des containers
          - Image de la liaison des noeuds :
        - 2. Démarrage et configuration des adresses IP
          - a) LAN
          - b) WAN
          - c) DMZ
          - d) Pare-feu
          - Subnets utilisés
          - Image de la topologie :
      - 3. Configuration des servers
        - 3.1 public-server
        - 3.2 internal-server
        - 3.3 internet
    - 3. **Configuration des services et comportements attendus**
      - 3.1. **Pare-feu**

- - Autoriser HTTP, HTTPS, SSH vers la DMZ (depuis WAN et LAN)
- - Bloquer tout autre port ou protocole par défaut
- - Bloquer l'accès du WAN vers le LAN
- - Autoriser le LAN à sortir vers l'extérieur via NAT
- - Mise en place d'un serveur DNS (Dnsmasq)
  - 1. Configuration de Dnsmasq
  - 2. Ajout d'entrées DNS locales
  - 3. Démarrage du service
  - 4. Configuration des clients DNS
  - 5. Test de la résolution DNS
- - Bloquer les sites pornographiques (SquidGuard)
  - 1. Création d'un fichier de configuration personnalisé pour Squid
  - 2. Configuration de SquidGuard
  - 3. Création de la liste noire
  - 4. Redirection du trafic avec iptables
  - 5. Finalisation de la configuration
  - 5.1 Vérification de la configuration
  - 5.2 Vérification de SquidGuard
  - 6. Test du filtrage
- - Bloquer les réseaux sociaux pendant les heures de travail
  - 1. Mise à jour de la configuration de SquidGuard
  - 2. Création de la liste noire des réseaux sociaux
  - 3. Finalisation et Test
- - Limiter la bande passante avec TC (Traffic Control)
  - 1. Conversion et terminologie
  - 2. Justification
  - 3. Fonctionnement de TC avec HTB
  - 4. Explication du script de limitation de bande passante
  - 5. Tests fonctionnels et vérification
- 3.2. **DMZ**
- 3.3. **Surveillance IDS avec Snort**
  - Detecter et journaliser les tentatives d'attaque et de scan
    - 1. Configuration de base de Snort
    - 2. Modification des variables réseau
    - 3. Création de règles de détection personnalisées
    - 4. Configuration de la journalisation
    - 5. Démarrage de Snort avec la nouvelle configuration

- Résultats des tests
  - Connectivité et isolation des réseaux
  - Filtrage de contenu
  - Qualité de service et limitation de bande passante
  - Détection d'intrusion
- Analyse et conclusion
  - Réalisation des objectifs
  - Difficultés rencontrées
  - Conclusion

Introduction

L'objectif de ce travail pratique est de mettre en œuvre une architecture réseau sécurisée intégrant une zone démilitarisée (DMZ), en utilisant les outils GNS3 et Docker. Ce projet s'inscrit dans le cadre de l'apprentissage des techniques de défense périmétrique d'un système d'information. Il permet de concevoir, configurer et analyser une topologie réseau complète, comprenant la mise en place d'une DMZ, la gestion des règles de filtrage et de NAT, le déploiement de solutions de détection/prévention d'intrusion ainsi que l'analyse des logs pour valider la politique de sécurité. À travers la réalisation de ce TP, les compétences en conception d'architectures sécurisées, en administration réseau et en cybersécurité seront renforcées.

Méthodologie

Environnement

- Ubuntu 24.04 Lts sous WSL 2.5.9
- Docker 28.3.3
- GN3 2.2.54

La méthodologie suivie pour la réalisation de ce travail pratique s'articule autour des étapes suivantes

1. **Création des images Docker**

La première étape a consisté à créer les images Docker nécessaires pour simuler l'ensemble des machines de notre architecture DMZ. Plusieurs Dockerfiles ont été développés, chacun correspondant à un rôle spécifique dans l'infrastructure :

1.1 Client LAN (`ubuntu_client`)

Cette image simule une machine cliente dans le réseau LAN interne:

```
FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    lynx \
    iputils-ping \
    traceroute \
    curl \
    wget \
    nano \
    openssh-client \
    iproute2 \
    dnsutils
```

1.2 Pare-feu (`ubuntu_firewall`)

Cette image centralise toutes les fonctions de sécurité réseau :

```
FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    iptables \
    iproute2 \
    squid \
    squidguard \
    squid-cgi \
    dnsmasq \
    iputils-ping \
    nano
```

1.3 Serveur Web DMZ (`ubuntu_webserver`)

Cette image héberge le serveur web situé dans la DMZ et intègre des outils de supervision et de sécurité :

```
FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    apache2 \
    iputils-ping \
    curl \
    wget \
    nano \
    openssh-client \
    openssh-server \
    iproute2 \
    apache2-utils \
    tcpdump \
    snort
```

1.4 Serveur Web LAN (`ubuntu_lan_webserver`)

Cette image simule un serveur web interne au LAN :

```
FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    apache2 \
    iputils-ping \
    curl \
    wget \
```

```
    nano \
    iproute2 \
    apache2-utils
```

1.5 WAN (`ubuntu_wan`)

Cette image simule une machine externe sur Internet :

```
FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    hping3 \
    lynx \
    iputils-ping \
    traceroute \
    curl \
    wget \
    nano \
    openssh-client \
    iproute2 \
    apache2
```

Toutes ces images utilisent Ubuntu 20.04 comme base et incluent la variable d'environnement `DEBIAN_FRONTEND=noninteractive` pour automatiser l'installation des paquets sans interaction utilisateur.

2. **Mise en place de la topologie**

2.1 Création des containers dockers dans gns3

firewall

- docker image : ubuntu_firewall
- interfaces : 3
- console : telnet

public-server

- docker image : ubuntu_webserver
- interfaces : 1
- console : telnet

internal-server

- docker image : ubuntu_lan_webserver
- interfaces : 1
- console : telnet

internal-desktop

- docker image : ubuntu_client
- interfaces : 1
- console : telnet

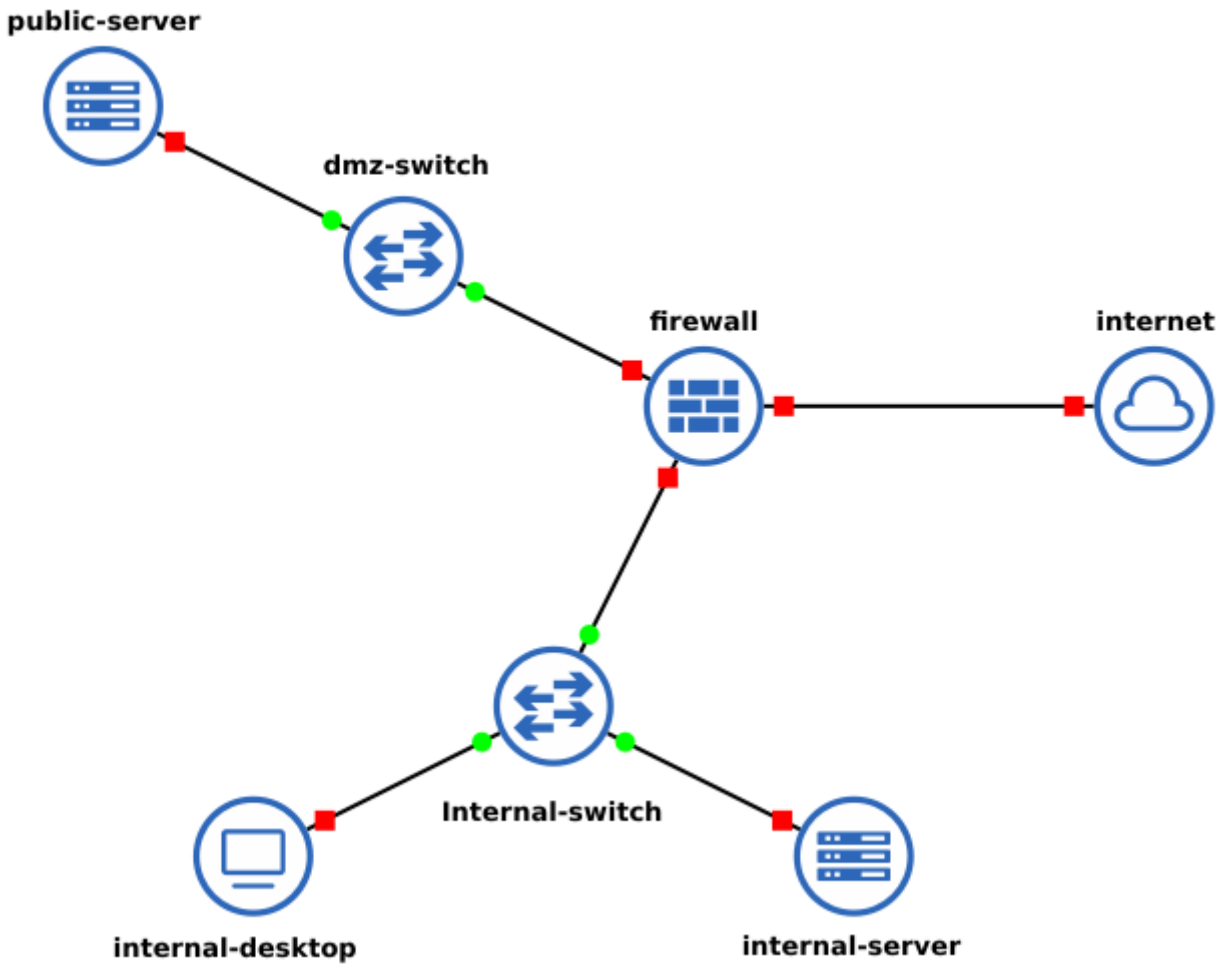
internet

- docker image : ubuntu_wan
- interfaces : 1
- console : telnet

2.2 Mise en place de la topologie dans gns3

1. Instanciation et liaison des containers

On commence par instancier les containers et les relier comme dans la figure ci-dessous :



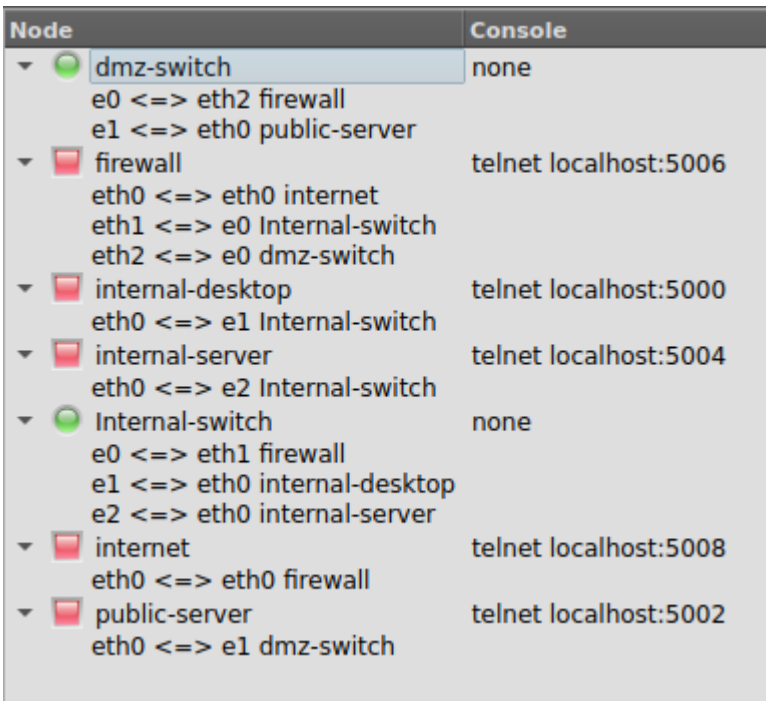
Structure réseau :

- **dmz-switch**
  - e0 ←→ eth2 firewall

    - e1 ←→ eth0 public-server

- **firewall**
    - eth0 ←→ e0 internet
    - eth1 ←→ e0 internal-switch
    - eth2 ←→ e1 dmz-switch
- **internal-desktop**
    - eth0 ←→ e1 internal-switch
- **internal-server**
    - eth0 ←→ e2 internal-switch
- **internal-switch**
    - e0 ←→ eth1 firewall
    - e1 ←→ eth0 internal-desktop
    - e2 ←→ eth0 internal-server
- **internet**
    - eth0 ←→ eth0 firewall
- **public-server**
    - eth0 ←→ e1 dmz-switch

Image de la liaison des noeuds :



2. Démarrage et configuration des adresses IP

Après le démarrage de toutes les instances, on configure leurs adresses IP comme suit :

a) LAN

- **internal-desktop**

```
ip a add 19.4.3.1/29 dev eth0
ip route add default via 19.4.3.3 dev eth0
echo "nameserver 19.4.3.3" > /etc/resolv.conf
```

- **internal-server**

```
ip a add 19.4.3.2/29 dev eth0
ip route add default via 19.4.3.3 dev eth0
echo "nameserver 19.4.3.3" > /etc/resolv.conf
```

b) WAN

- **internet**

```
ip a add 19.4.3.17/29 dev eth0
ip route add default via 19.4.3.18 dev eth0
```

c) DMZ

- **public-server (DMZ)**

```
ip a add 19.4.3.9/29 dev eth0
ip route add default via 19.4.3.11 dev eth0
```

d) Pare-feu

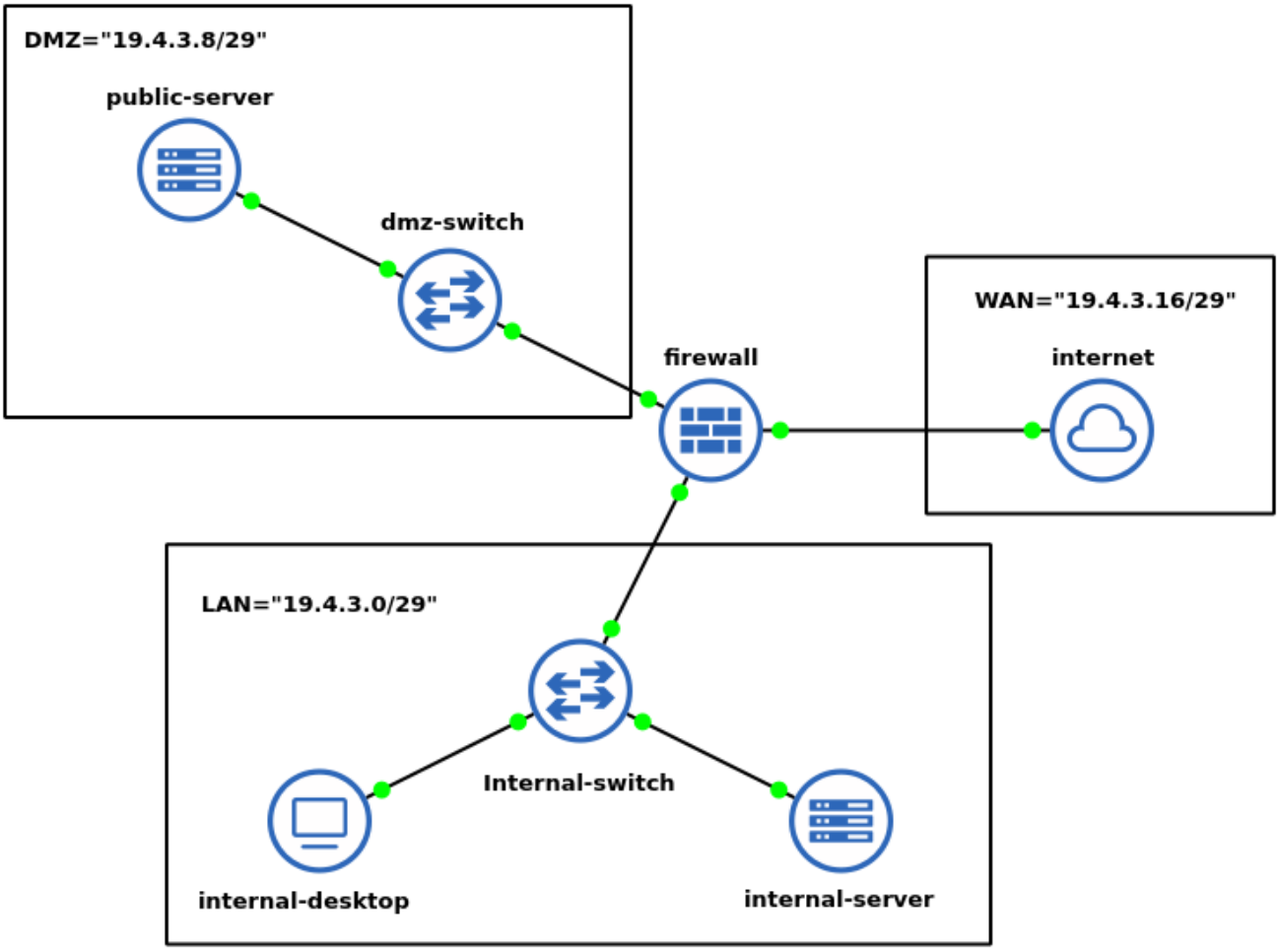
- **firewall**

```
ip a add 19.4.3.3/29 dev eth1
ip a add 19.4.3.11/29 dev eth2
ip a add 19.4.3.18/29 dev eth0
sysctl -w net.ipv4.ip_forward=1
```

Subnets utilisés

```
WAN="19.4.3.16/29"
LAN="19.4.3.0/29"
DMZ="19.4.3.8/29"
```

Image de la topologie :



3. Configuration des servers

3.1 public-server

```
echo "ServerName localhost" | tee /etc/apache2/conf-available/servername.conf
a2enconf servername
service apache2 restart

cd /var/www/html/

echo "<html>
<head>
   <title>Serveur Apache dans Docker DMZ</title>
</head>
```

```
<body>
   <h1>Bienvenue sur le serveur publique</h1>
</body>
</html>" > /var/www/html/index.html

a2enmod ssl
a2enmod rewrite

mkdir -p /etc/ssl/private
mkdir -p /etc/ssl/certs

openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
   -keyout /etc/ssl/private/apache-selfsigned.key \
   -out /etc/ssl/certs/apache-selfsigned.crt \
   -subj "/C=FR/ST=State/L=City/O=Organization/OU=OrgUnit/CN=webserver"

cat > /etc/apache2/sites-available/default-ssl.conf << 'EOF'
<IfModule mod_ssl.c>
   <VirtualHost *:443>
       DocumentRoot /var/www/html

       SSLEngine on
       SSLCertificateFile /etc/ssl/certs/apache-selfsigned.crt
       SSLCertificateKeyFile /etc/ssl/private/apache-selfsigned.key

       <FilesMatch "\.(cgi|shtml|phtml|php)$">
           SSLOptions +StdEnvVars
       </FilesMatch>
   </VirtualHost>
</IfModule>
EOF

a2ensite default-ssl

service apache2 restart

mkdir -p /var/run/sshd
ssh-keygen -A
echo "PasswordAuthentication yes" >> /etc/ssh/sshd_config
echo "PermitRootLogin yes" >> /etc/ssh/sshd_config
useradd -m -s /bin/bash user
echo "user:root" | chpasswd
service ssh start
```

3.2 internal-server

```
 echo "ServerName localhost" | tee /etc/apache2/conf-available/servername.conf
 a2enconf servername
 service apache2 restart
```

```
cd /var/www/html/

echo "<html>
<head>
    <title>Serveur Apache dans Docker Interne</title>
</head>
<body>
    <h1>Bienvenue sur le serveur interne</h1>
</body>
</html>" > /var/www/html/index.html
```

3.3 internet

```
echo "ServerName localhost" | tee /etc/apache2/conf-available/servername.conf
a2enconf servername
service apache2 restart

cd /var/www/html/

echo "<html>
<head>
    <title>Mon serveur Apache dans Docker du WAN</title>
</head>
<body>
    <h1>Bienvenue sur le WAN</h1>
</body>
</html>" > /var/www/html/index.html
```

3. **Configuration des services et comportements attendus**

3.1. Pare-feu

On definit tout d'abord les variables à utiliser :

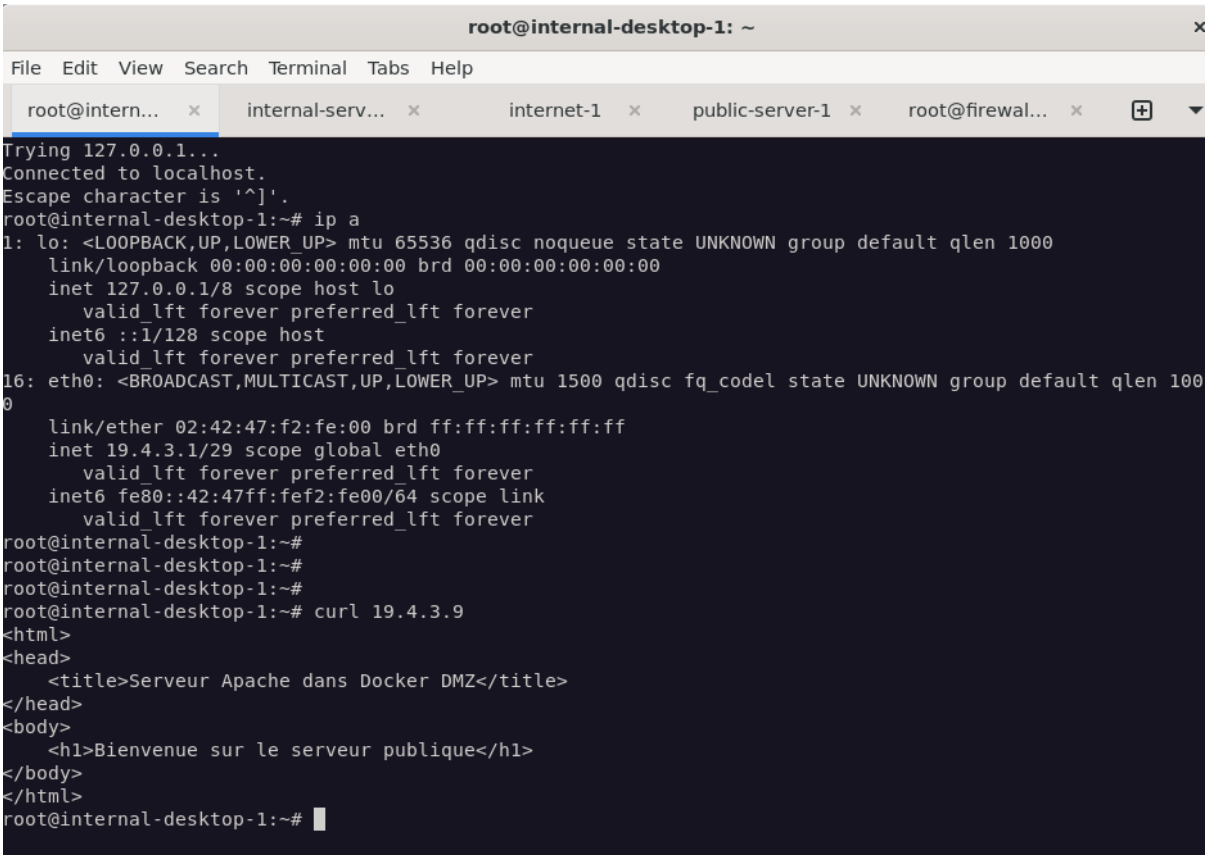
```
WAN="19.4.3.16/29"
LAN="19.4.3.0/29"
DMZ="19.4.3.8/29"
HTTP="80"
HTTPS="443"
SSH="22"

WAN_IFACE="eth0"
LAN_IFACE="eth1"
DMZ_IFACE="eth2"
```

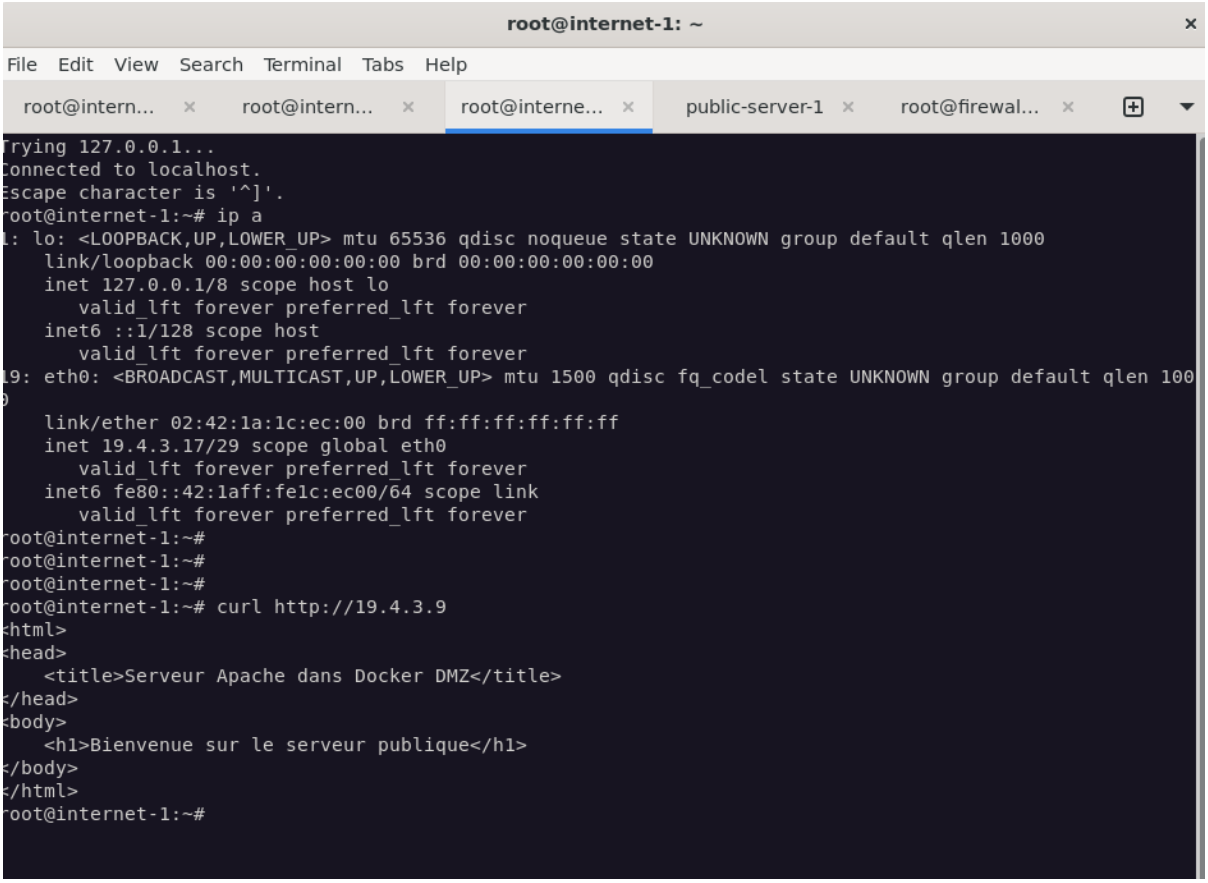
Etant donné qu'on a crée l'image docker à partir d'une image **ubuntu vierge** et qu'on a **nous meme installé les packets necessaires**, `iptables` **ne contient encore aucune regle**.

- Autoriser HTTP, HTTPS, SSH vers la DMZ (depuis WAN et LAN)

```
 iptables -A FORWARD -p tcp -s $LAN,$WAN -d $DMZ -m multiport --dports
$HTTP,$HTTPS,$SSH -m conntrack --ctstate NEW,ESTABLISHED,RELATED -j ACCEPT
 iptables -A FORWARD -s $DMZ -d $LAN,$WAN -m conntrack --ctstate
ESTABLISHED,RELATED -j ACCEPT
```

- Curl depuis LAN vers DMZ
  - 

- Curl depuis WAN vers DMZ



Justification :

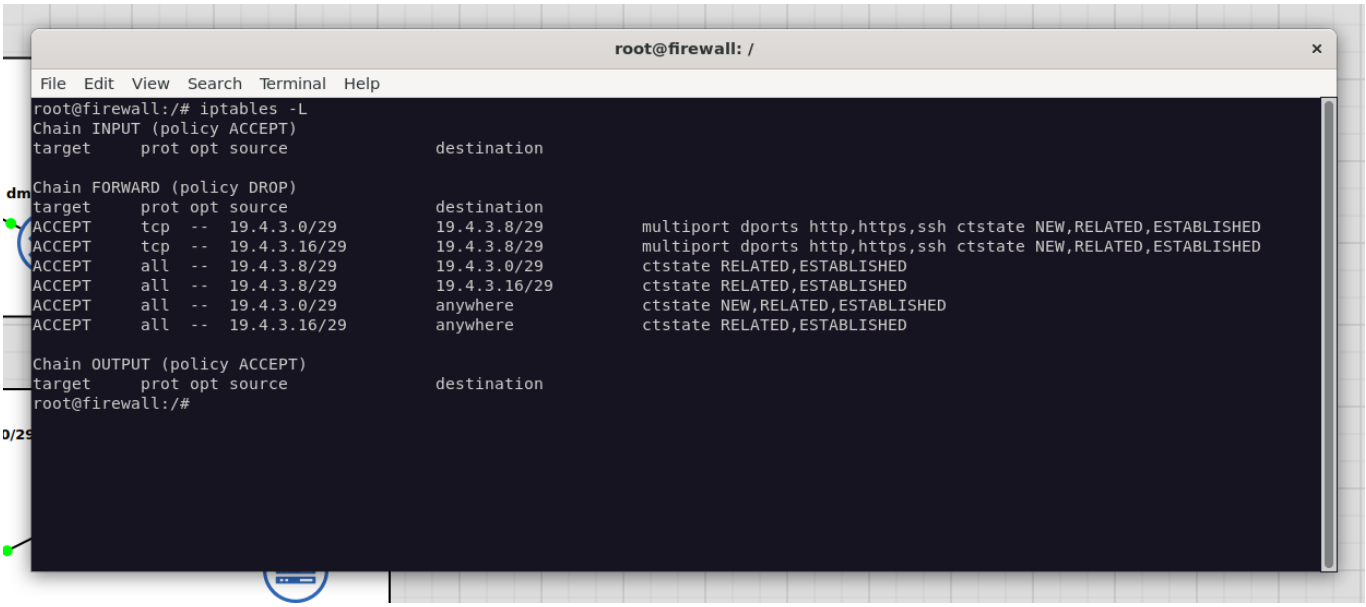
1. Première règle (`iptables -A FORWARD -p tcp -s $LAN,$WAN -d $DMZ...`) :
   - `-A FORWARD` ajoute la règle à la chaîne FORWARD qui traite les paquets routés entre interfaces
   - `-p tcp` filtre uniquement le protocole TCP
   - `-s $LAN,$WAN` spécifie les sources autorisées (réseau local et Internet)
   - `-d $DMZ` cible uniquement les paquets à destination de la DMZ
   - `-m multiport --dports $HTTP,$HTTPS,$SSH` utilise le module multiport pour spécifier efficacement plusieurs ports de destination (80, 443, 22)
   - `-m conntrack --ctstate NEW,ESTABLISHED,RELATED` active le module de suivi de connexion et autorise : les nouvelles connexions (NEW), les paquets appartenant à une connexion déjà établie (ESTABLISHED), et les paquets liés à une connexion existante comme dans le FTP passif (RELATED)
   - `-j ACCEPT` applique l'action d'acceptation pour les paquets correspondant à tous les critères ci-dessus

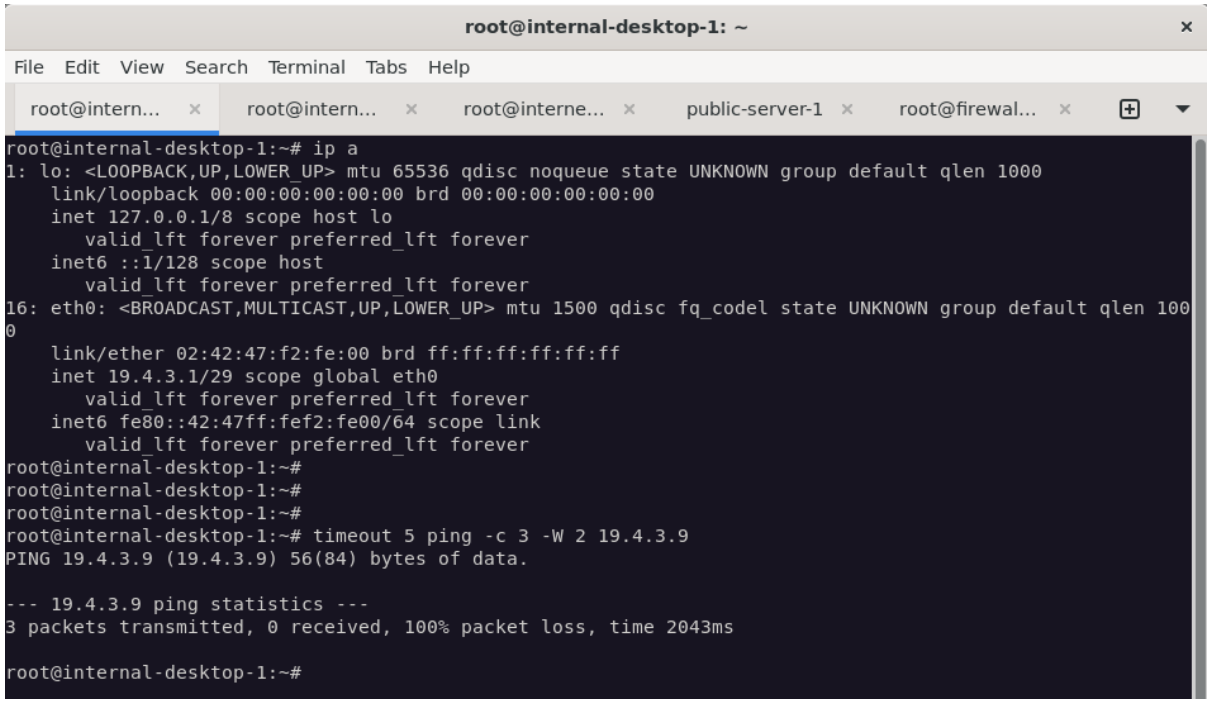
2. Seconde règle (`iptables -A FORWARD -s $DMZ -d $LAN,$WAN...`) :
   - Cette règle complémentaire autorise uniquement les réponses depuis la DMZ vers le LAN ou le WAN
   - L'absence de l'état `NEW` dans `--ctstate` est cruciale : elle empêche les serveurs de la DMZ d'initier des connexions vers l'intérieur ou l'extérieur, limitant ainsi la portée d'une éventuelle compromission

- Bloquer tout autre port ou protocole par défaut

```
iptables -P FORWARD DROP
```

Justification : Cette règle applique une politique de sécurité par défaut restrictive (deny-by-default), un principe fondamental dans la conception de pare-feu. En bloquant tout trafic non explicitement autorisé, nous réduisons considérablement la surface d'attaque. Cette approche est essentielle car elle garantit que seules les connexions précédemment définies comme légitimes sont autorisées à traverser le pare-feu, tandis que toute tentative d'exploitation de services non autorisés est automatiquement bloquée. Cela nous protège également contre les risques liés aux services qui pourraient être activés par erreur ou aux nouvelles vulnérabilités découvertes sur des ports non surveillés.



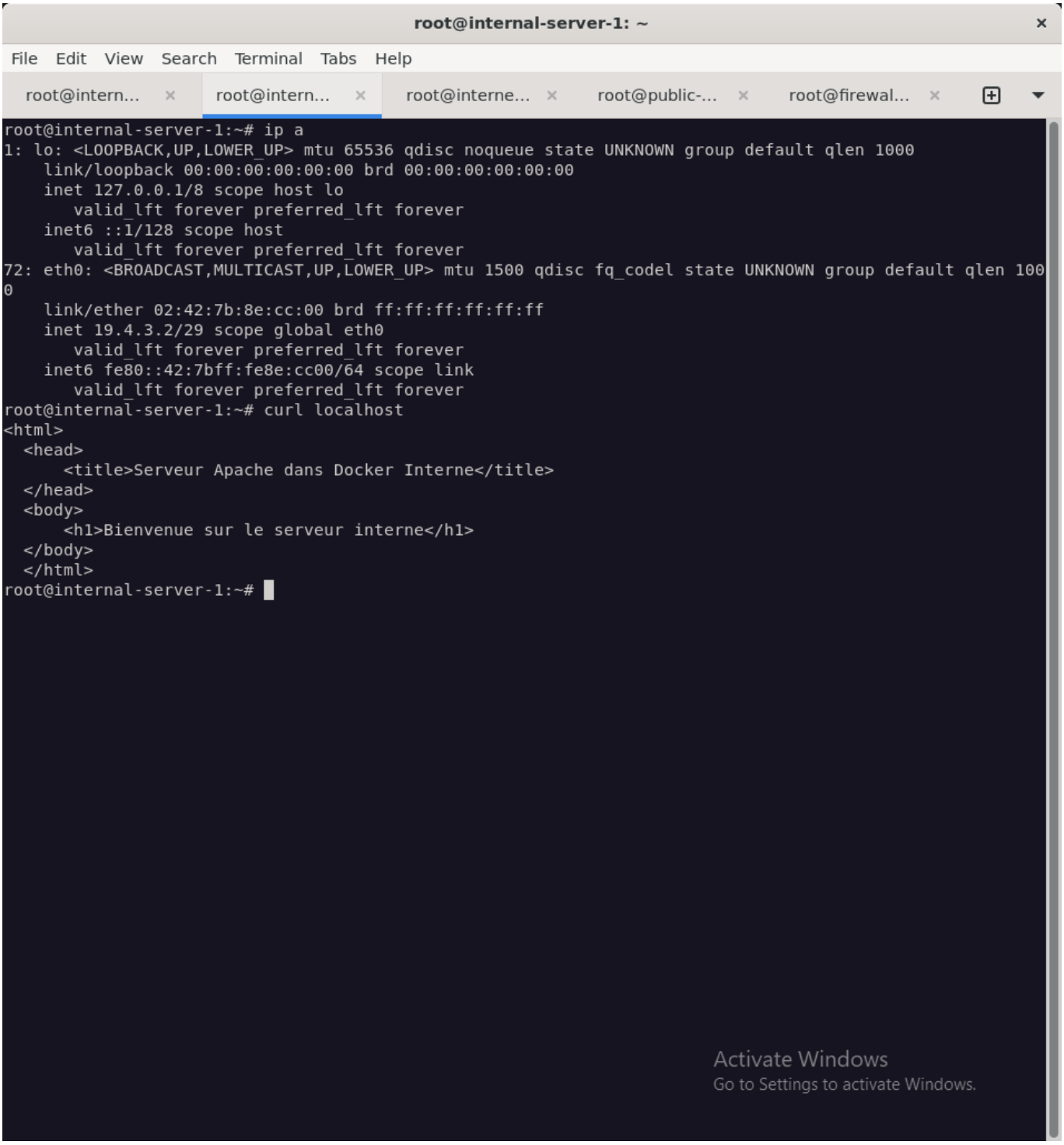
- ping depuis LAN vers DMZ :
  - 

Justification : Le ping a echoué car le protocol utilisé pour le ping est bloqué

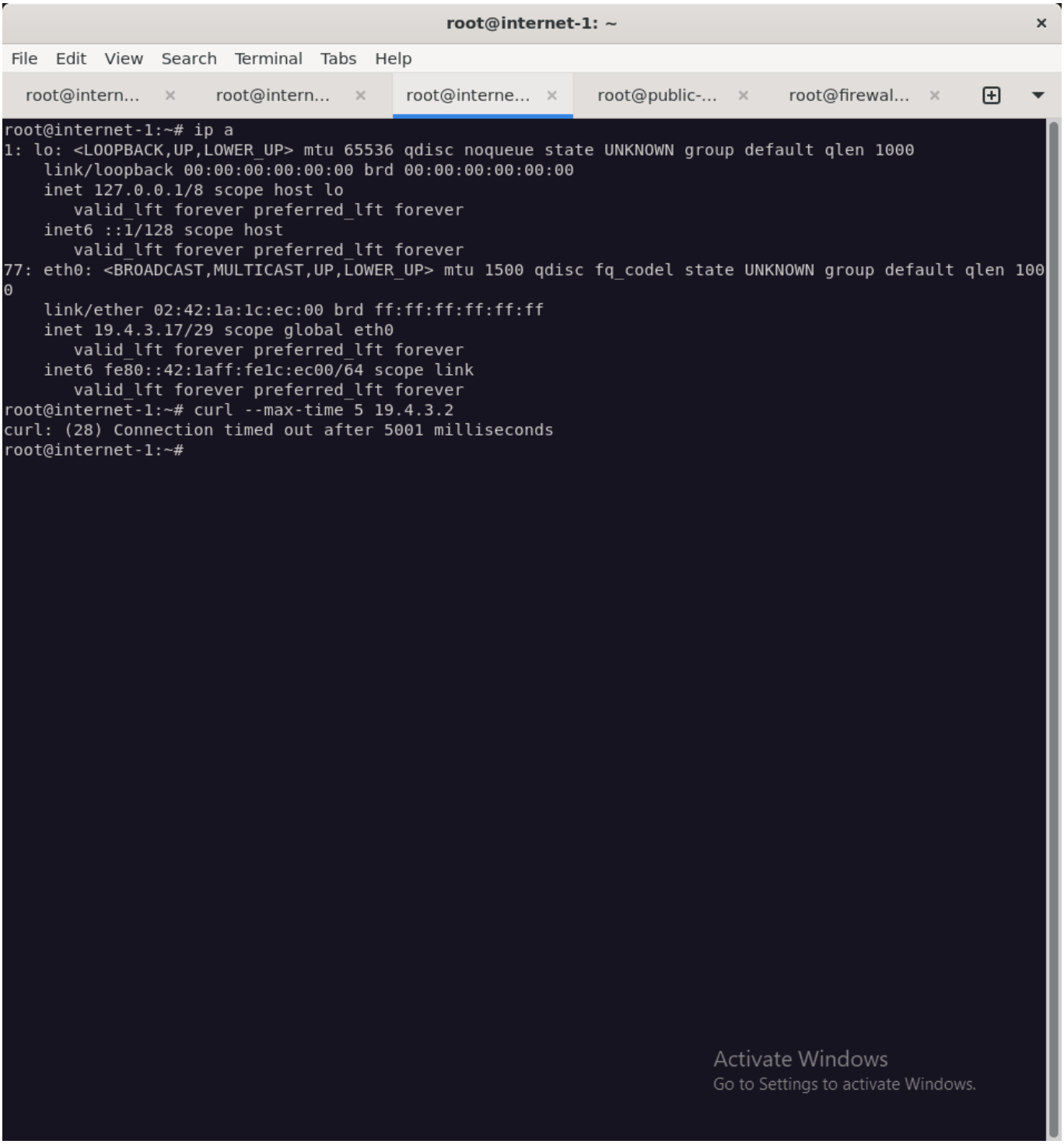
- Bloquer l'accès du WAN vers le LAN

```
iptables -A FORWARD -p tcp -s $WAN -d $LAN -m conntrack --ctstate NEW -j DROP
iptables -A FORWARD -p tcp -s $WAN -d $LAN -m conntrack --ctstate
ESTABLISHED,RELATED -j ACCEPT
```

curl du server LAN local vers lui meme :

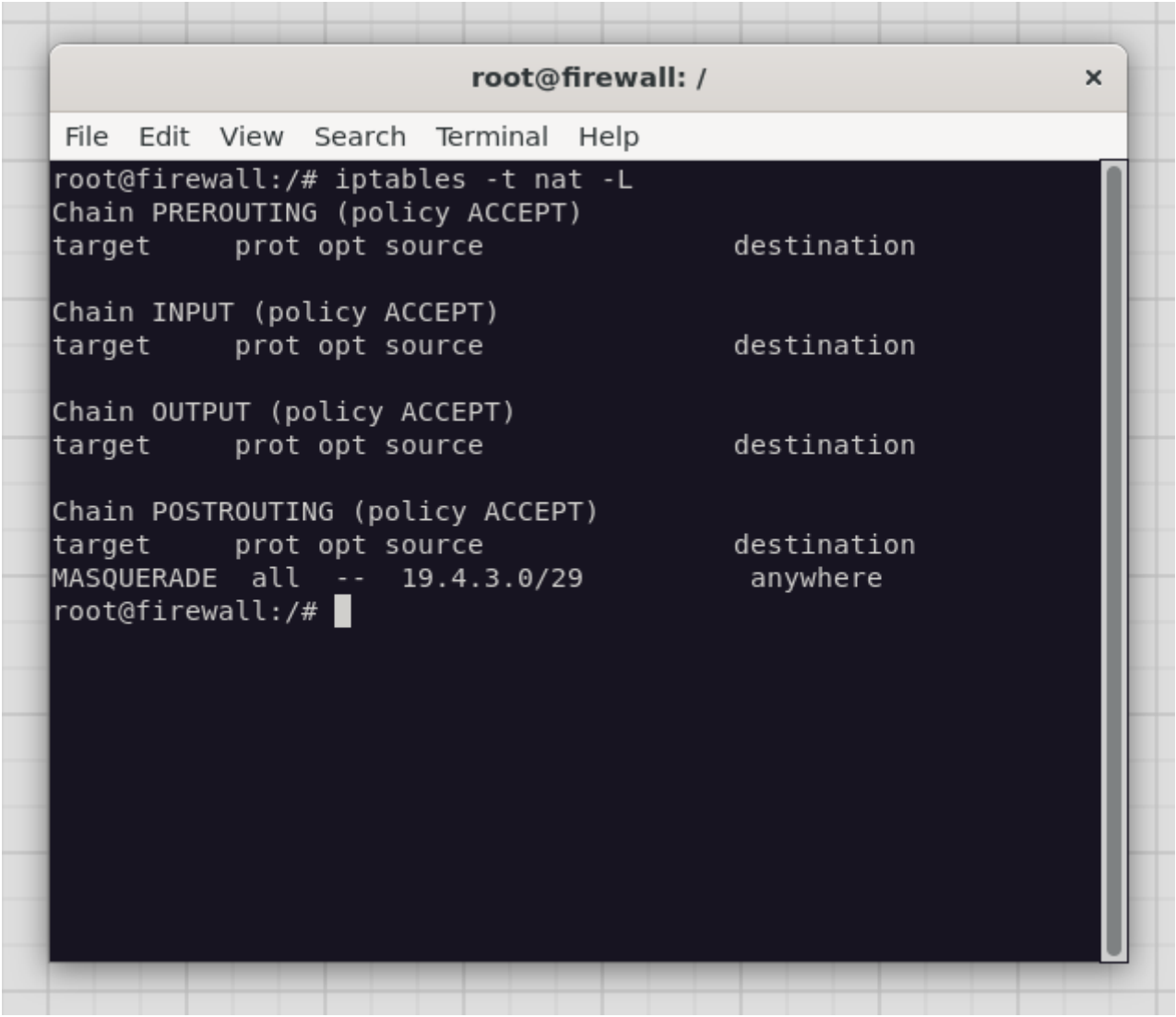


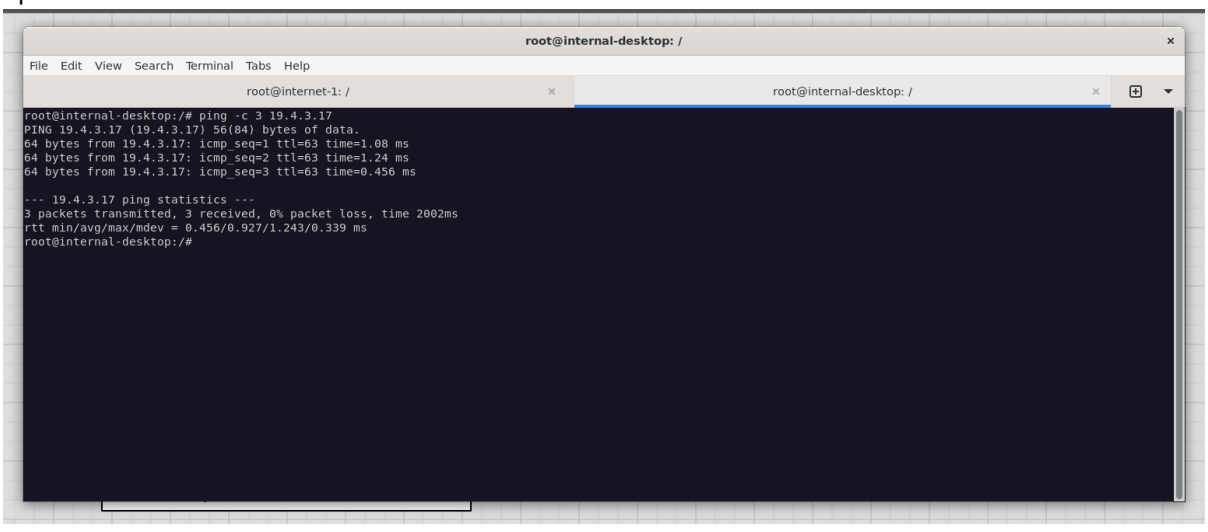
curl depuis le WAN vers me server LAN local :

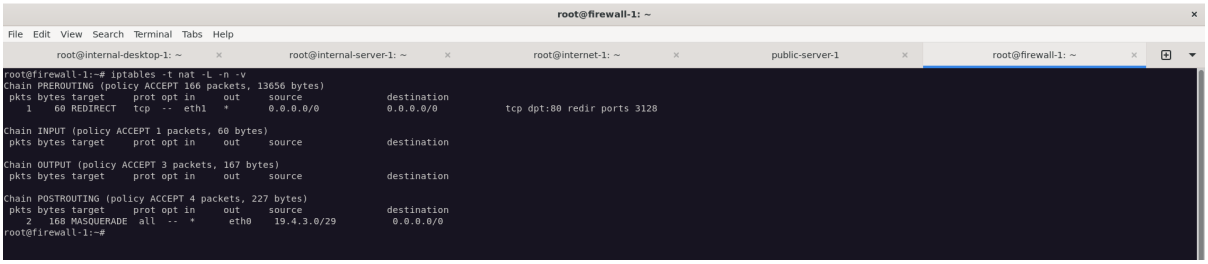


- Autoriser le LAN à sortir vers l'extérieur via NAT

```
 iptables -t nat -A POSTROUTING -s $LAN -o $WAN_IFACE -j MASQUERADE
 iptables -A FORWARD -s $LAN -o $WAN_IFACE -m conntrack --ctstate
NEW,ESTABLISHED,RELATED -j ACCEPT
 iptables -A FORWARD -s $WAN -i $WAN_IFACE -m conntrack --ctstate
ESTABLISHED,RELATED -j ACCEPT
```



- ping depuis LAN vers WAN :
  - 

- affichage des packets dans le firewall :
  - 

Justification : Ces trois règles fonctionnent ensemble pour permettre la communication du réseau LAN vers l'extérieur tout en maintenant la sécurité :

1. La première règle (`MASQUERADE`) agit sur la table NAT et manipule les paquets sortants. Elle remplace l'adresse IP source des paquets provenant du LAN par l'adresse IP publique du pare-feu sur son interface WAN. Cette substitution permet aux machines du LAN (qui utilisent des adresses privées) de communiquer avec Internet.
2. La deuxième règle autorise le transfert des paquets du LAN vers le WAN. L'option `--ctstate NEW,ESTABLISHED,RELATED` est cruciale : elle permet aux machines du LAN d'initier de nouvelles connexions (`NEW`) et de maintenir les connexions déjà établies (`ESTABLISHED`) ou liées à une connexion existante (`RELATED`).
3. La troisième règle permet le retour des paquets de réponse depuis le WAN vers le LAN, mais uniquement pour des connexions déjà établies ou associées (`ESTABLISHED,RELATED`). Cette restriction empêche l'initiation de connexions depuis l'extérieur, renforçant ainsi la sécurité du réseau interne.

- Mise en place d'un serveur DNS (Dnsmasq)

Pour simplifier la résolution de noms dans notre topologie, nous allons configurer `dnsmasq` sur le pare-feu. Ce service agira comme un serveur DNS pour le LAN, résolvant les noms de domaine de notre réseau local et transférant les autres requêtes à des serveurs DNS externes (simulés ou réels).

Justification : L'implémentation d'un serveur DNS local avec Dnsmasq sur notre pare-feu etant donné que notre environnement ne dispose pas d'access à internet et donc on ne peut pas utiliser les DNS standards comme ceux de google ou cloudflare.

1. Configuration de Dnsmasq

Créez un fichier de configuration pour `dnsmasq` sur le pare-feu. Nous allons le configurer pour qu'il écoute uniquement sur l'interface du LAN et utilise le fichier `/etc/hosts` du pare-feu comme source pour les noms locaux.

```
# Sur le pare-feu
cat > /etc/dnsmasq.conf << 'EOF'
interface=eth1
no-resolv
server=8.8.8.8
expand-hosts
domain=lan.local
EOF
```

2. Ajout d'entrées DNS locales

Pour que notre test de filtrage fonctionne sans modifier le fichier `/etc/hosts` de chaque client, nous pouvons ajouter l'entrée directement dans le fichier `/etc/hosts` du pare-feu. `dnsmasq` le servira ensuite à tout le LAN.

```
# Sur le pare-feu
echo "19.4.3.17 pornhub.com" >> /etc/hosts
echo "19.4.3.17 facebook.com" >> /etc/hosts
echo "19.4.3.17 twitter.com" >> /etc/hosts
echo "19.4.3.9 webserver.dmz" >> /etc/hosts
echo "19.4.3.2 internal.lan" >> /etc/hosts
```

3. Démarrage du service

```
# Sur le pare-feu
service dnsmasq start
```

4. Configuration des clients DNS

Nous avons déjà mis à jour la configuration des machines du LAN à l'étape 2.2 pour qu'elles utilisent le pare-feu (`19.4.3.3`) comme serveur DNS.

5. Test de la résolution DNS

Depuis `internal-desktop`, vous pouvez maintenant utiliser `nslookup` ou `dig` pour vérifier que la résolution de noms fonctionne.

```
# Sur internal-desktop
nslookup pornhub.com
```

La réponse devrait être `19.4.3.17`.

- Bloquer les sites pornographiques (SquidGuard)

Pour bloquer l'accès à des sites spécifiques, nous utiliserons Squid comme proxy transparent et SquidGuard comme filtre d'URL. Le trafic HTTP du LAN sera redirigé vers Squid, qui le passera à SquidGuard pour inspection.

1. Création d'un fichier de configuration personnalisé pour Squid

Pour éviter les conflits avec la configuration par défaut, nous allons créer un fichier séparé qui contiendra nos directives. Squid lira ce fichier et appliquera nos règles.

Créez le fichier `/etc/squid/conf.d/00-custom.conf` sur le pare-feu :

```
cat > /etc/squid/conf.d/00-custom.conf << 'EOF'
http_port 3128 intercept
http_port 127.0.0.1:3129
```

```
acl localnet src 19.4.3.0/29
http_access allow localnet
http_access deny all

url_rewrite_program /usr/bin/squidGuard -c /etc/squidguard/squidGuard.conf
url_rewrite_children 5

cache deny all
EOF
```

2. Configuration de SquidGuard

Créez et configurez le fichier `/etc/squidguard/squidGuard.conf` pour définir les listes de blocage.

```
cat >/etc/squidguard/squidGuard.conf<< 'EOF'
dbhome /var/lib/squidguard/db

logdir /var/log/squid

src lan {
    ip 19.4.3.0/29
}

dest porn {
    domainlist porn/domains
    urllist porn/urls
}

acl {
    lan {
        pass !porn all
        redirect http://19.4.3.3/cgi-bin/squidGuard.cgi?
clientaddr=%a&clientname=%n&clientuser=%i&clientgroup=%s&targetgroup=%t&url=%u
    }
    default {
        pass none
        redirect http://19.4.3.3/cgi-bin/squidGuard.cgi?
clientaddr=%a&clientname=%n&clientuser=%i&clientgroup=%s&targetgroup=%t&url=%u
    }
}
EOF
```

3. Création de la liste noire

Nous devons créer les fichiers contenant les domaines et URL à bloquer.

```
mkdir -p /var/lib/squidguard/db/porn

cat > /var/lib/squidguard/db/porn/domains << 'EOF'
pornhub.com
xvideos.com
xnxx.com
redtube.com
youporn.com
EOF

touch /var/lib/squidguard/db/porn/urls
```

4. Redirection du trafic avec iptables

Ajoutez une règle `iptables` sur le pare-feu pour rediriger tout le trafic HTTP (port 80) provenant du LAN vers le port de Squid (3128).

Rediriger le trafic HTTP du LAN vers Squid

```
iptables -t nat -A PREROUTING -i $LAN_IFACE -p tcp --dport 80 -j REDIRECT --to-port 3128
```

Justification du fonctionnement de la règle : Cette règle NAT utilise la chaîne `PREROUTING` qui intercepte les paquets avant qu'ils ne soient routés. Le paramètre `-i $LAN_IFACE` spécifie que seul le trafic entrant sur l'interface LAN est concerné. Le filtre `-p tcp --dport 80` cible spécifiquement le trafic HTTP standard. L'action `REDIRECT --to-port 3128` redirige ce trafic vers le port local où Squid écoute, sans que les clients ne soient conscients de cette redirection.

5. Finalisation de la configuration

Compilez la base de données de SquidGuard, ajustez les permissions et redémarrez les services.

5.1 Vérification de la configuration

Avant de redémarrer Squid, il est crucial de vérifier que les fichiers de configuration ne contiennent pas d'erreurs de syntaxe.

```
squid -k parse
```

Si cette commande ne retourne aucune erreur, vous pouvez continuer. Sinon, vous devrez corriger les erreurs indiquées dans `squid.conf` avant de poursuivre.

5.2 Vérification de SquidGuard

Si la configuration de Squid est valide mais que le service ne démarre toujours pas, le problème vient probablement de SquidGuard.

```
squidGuard -b -d -C all

chown -R proxy:proxy /var/lib/squidguard/db
chown -R proxy:proxy /var/log/squid
chmod -R 755 /var/lib/squidguard/db

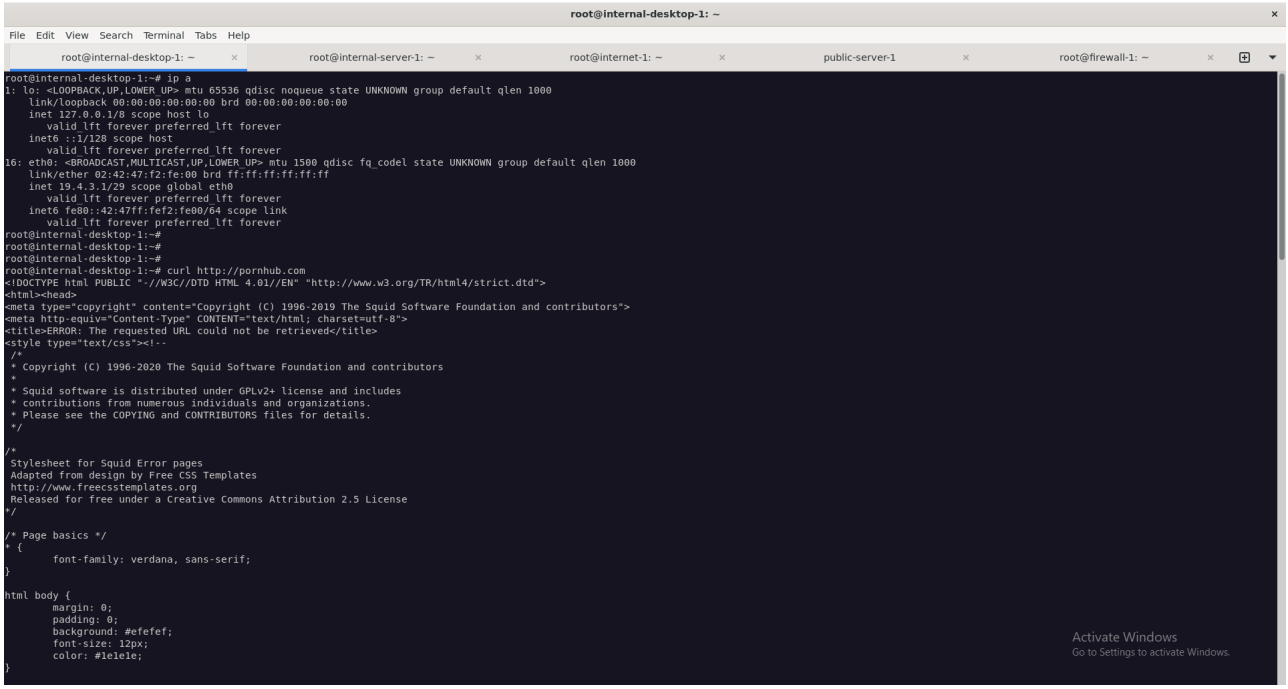
service squid restart
```

6. Test du filtrage

Dans notre topologie simulée, les machines du LAN n'ont pas accès à un serveur DNS public pour résoudre les noms de domaine comme `pornhub.com`. Pour pouvoir tester notre configuration, nous avons deja utilisé `dnsmasq`. Il ne nous reste plus qu'à executer la commande :

```
curl http://pornhub.com
```

- Affichage de la page de blocage par defaut de squid apres avoir executé le `curl http://pornhub.com`



- Bloquer les réseaux sociaux pendant les heures de travail

Nous utiliserons la fonctionnalité de gestion du temps de SquidGuard.

1. Mise à jour de la configuration de SquidGuard

Modifiez le fichier `/etc/squidguard/squidGuard.conf` pour y inclure une définition des plages horaires et une nouvelle catégorie de sites à bloquer ("social"). La section `acl` est également mise à jour pour appliquer des règles différentes en fonction de l'heure.

Le nouveau contenu complet du fichier de configuration est le suivant :

```
cat >/etc/squidguard/squidGuard.conf<< 'EOF'
dbhome /var/lib/squidguard/db

logdir /var/log/squid

src lan {
    ip 19.4.3.0/29
}

time workhours {
    weekly mtwhf 08:00-12:00
    weekly mtwhf 14:00-18:00
}

dest porn {
    domainlist porn/domains
    urllist porn/urls
}

dest social {
    domainlist social/domains
    urllist social/urls
}

acl {
    lan within workhours {
        pass !porn !social all
        redirect http://19.4.3.3/cgi-bin/squidGuard.cgi?
clientaddr=%a&clientname=%n&clientuser=%i&clientgroup=%s&targetgroup=%t&url=%u
    }

    lan {
        pass !porn all
        redirect http://19.4.3.3/cgi-bin/squidGuard.cgi?
clientaddr=%a&clientname=%n&clientuser=%i&clientgroup=%s&targetgroup=%t&url=%u
    }

    default {
        pass none
        redirect http://19.4.3.3/cgi-bin/squidGuard.cgi?
clientaddr=%a&clientname=%n&clientuser=%i&clientgroup=%s&targetgroup=%t&url=%u
    }
}
EOF
```

2. Création de la liste noire des réseaux sociaux

Ajoutez la liste des domaines de réseaux sociaux à bloquer.

Créer le répertoire et la liste pour les réseaux sociaux, lister les domaines et creer le fichier contenant les urls (pour ne pas avoir d'erreur lors de la verofocation du ficher de configuration)

```
mkdir -p /var/lib/squidguard/db/social

cat > /var/lib/squidguard/db/social/domains << 'EOF'
facebook.com
twitter.com
instagram.com
tiktok.com
linkedin.com
snapchat.com
pinterest.com
EOF

touch /var/lib/squidguard/db/social/urls
```

3. Finalisation et Test

Après avoir modifié la configuration et ajouté la nouvelle liste, vous devez recompiler la base de données de SquidGuard et redémarrer le service, comme à l'étape 5.

Compiler la base de données (inclut maintenant la liste "social"), s'assurer que les permissions sont correctes et redemarrer squid

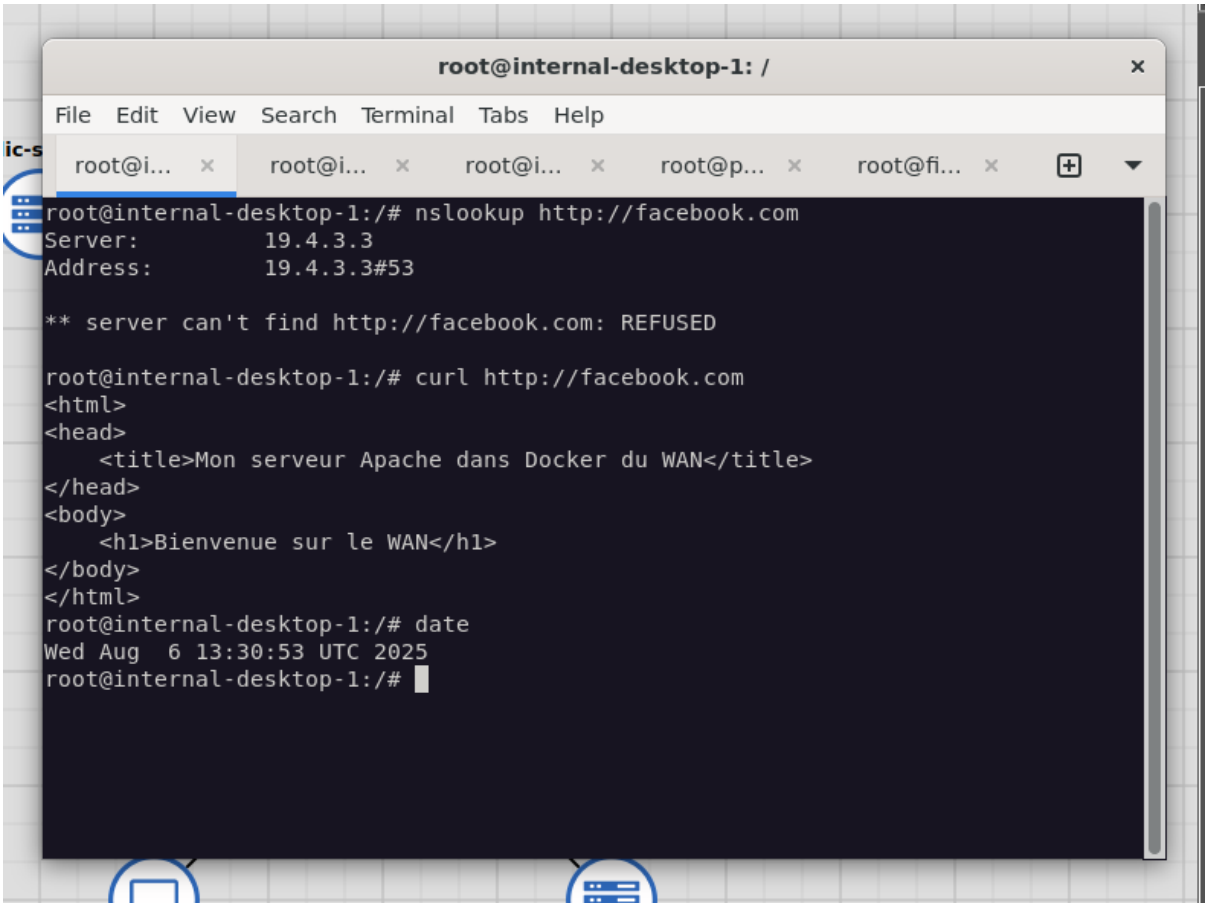
```
squidGuard -b -d -C all

service squid restart
```

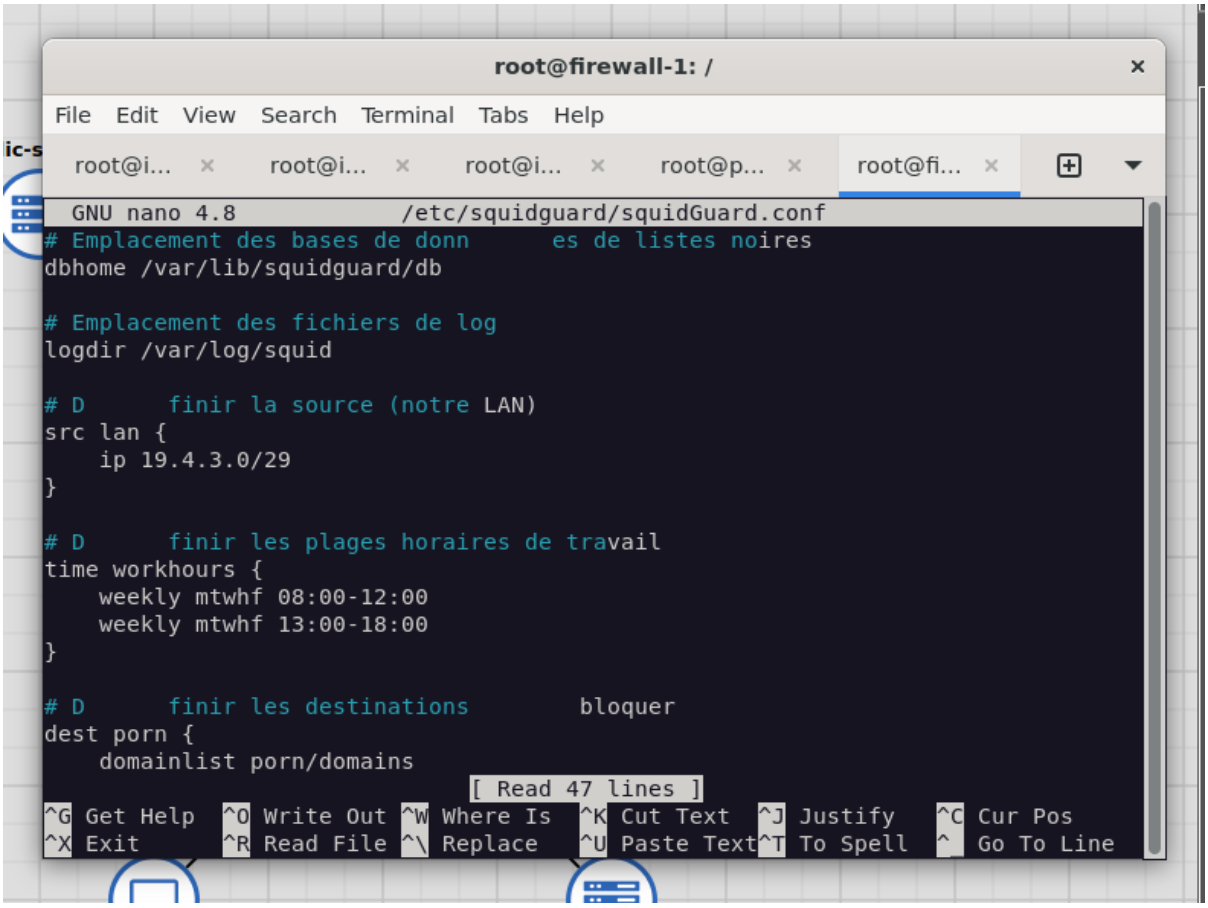
Pour tester, ajoutez les domaines des réseaux sociaux au fichier `/etc/hosts` du pare-feu (pour que `dnsmasq` les résolve), puis essayez d'y accéder depuis `internal-desktop` avec `curl`.

```
# Sur internal-desktop, pendant les heures de travail
curl http://facebook.com
```

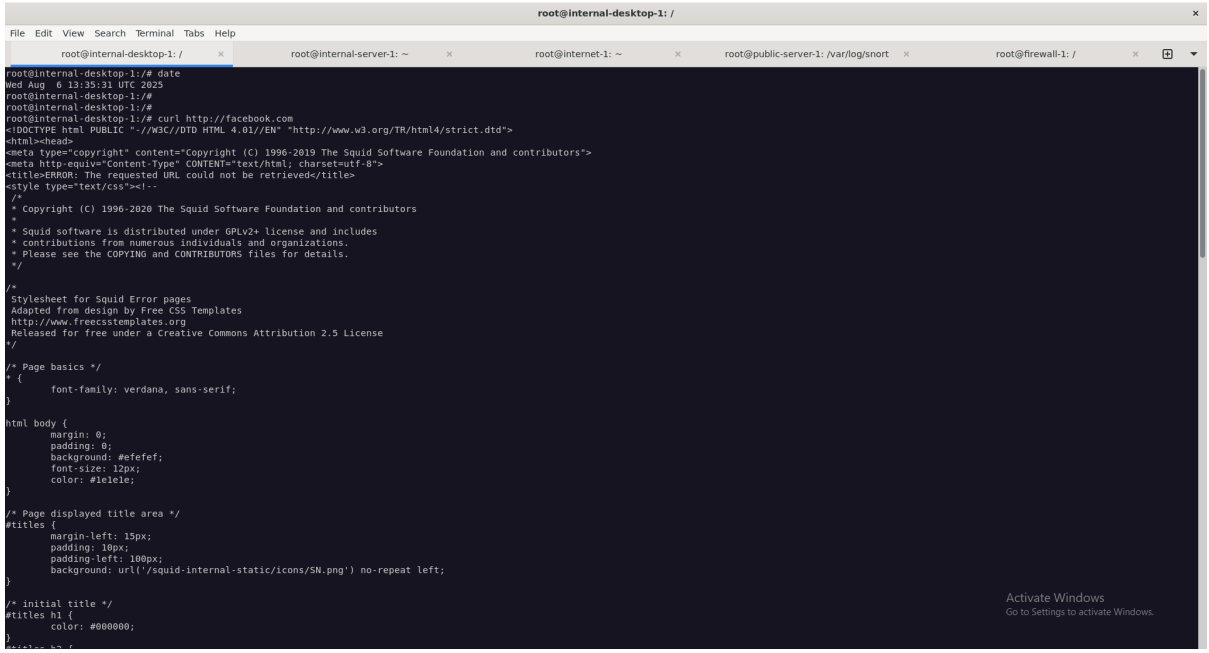
- curl depuis le LAN vers facebook à 13h :



- configuration de l'heure de travail à 13h pour le test :



- curl depuis le LAN vers facebook à durant les heures de travail:



- Limiter la bande passante avec TC (Traffic Control)

Objectif : plafonner chaque machine du LAN (19.4.3.0/29) à 500 Ko/s (≈ 4000 kbit/s) en download ET upload, de manière stable et équitable.

1. Conversion et terminologie

- 500 Ko/s = 500 * 1024 ≈ 512000 octets/s ≈ 4 096 000 bit/s ≈ 4000 kbit/s (arrondi).
- Plafond “par IP” signifie : chaque hôte du LAN ne dépassera pas 500 Ko/s, même s’il est seul.

2. Justification

La limitation de bande passante par hôte :

- Empêche la monopolisation des ressources (équité et prévisibilité).
- Décourage les téléchargements lourds non essentiels.

3. Fonctionnement de TC avec HTB

Principe :

- On applique un qdisc HTB racine sur chaque interface pertinente.
- On crée une classe par IP du LAN (sauf l’IP du firewall).
- Download (vers LAN) : classification par destination sur eth1.
- Upload (depuis LAN) : classification par source sur eth0 (vers WAN) et eth2 (vers DMZ).
- fq_codel par classe pour limiter la latence sous saturation.

> **Détails techniques**:
>
> - **HTB (Hierarchical Token Bucket)** : Algorithme de mise en file d'attente qui permet de créer une hiérarchie de classes de trafic, chacune avec ses propres limites de débit.

> - **fq_codel (Fair Queuing Controlled Delay)** : Algorithme de gestion de file d'attente qui réduit la latence en évitant le "bufferbloat" (accumulation excessive de paquets dans les tampons).
> - **tc (Traffic Control)** : Outil Linux permettant de configurer le contrôle de trafic du noyau.

Plage LAN (/29) :

- 19.4.3.0 réseau
- 19.4.3.1–19.4.3.6 hôtes
- 19.4.3.7 broadcast
- Firewall LAN : 19.4.3.3 (non limité)

Taux cible : 4000kbit (≈ 500 Ko/s).
Burst : petite tolérance contrôlée (ex: 25k) pour ne pas pénaliser les handshakes TCP/TLS.

4. Explication du script de limitation de bande passante

Le script suivant implémente une limitation de bande passante avancée en utilisant tc et HTB. Examinons les principales sections et commandes :

```
#!/usr/bin/env bash
set -euo pipefail

# Définition des variables réseau
LAN_BASE="19.4.3"                # Base de l'adressage IP du LAN
LAN_IF="eth1"   # Interface vers LAN (download shaping -> match dst)
WAN_IF="eth0"   # Interface vers WAN (upload shaping -> match src)
DMZ_IF="eth2"   # Interface vers DMZ (upload shaping -> match src)
FIREWALL_SUFFIX=3                # Dernier octet de l'IP du pare-feu (ne sera pas
limité)
RATE="4000kbit"                  # Taux de transfert cible (~500 Ko/s)
BURST="25k"                      # Tolérance de burst pour les pics courts
CEIL="$RATE"                     # Plafond (identique au taux de base)

# Plage d'adresses des hôtes LAN (dans un /29: .1 à .6 sont utilisables)
HOST_START=1
HOST_END=6

# Fonction de journalisation pour le suivi des opérations
log(){ echo "[apply_tc_per_ip] $*"; }

# Fonction qui crée un qdisc HTB racine sur une interface si nécessaire
# Cette fonction est idempotente (peut être appelée plusieurs fois sans effet
secondaire)
maybe_add_root() {
  local IF=$1
  # Vérifie si un qdisc HTB existe déjà sur l'interface
  if ! tc qdisc show dev "$IF" | grep -q "htb 1:" ; then
     log "Root HTB sur $IF"
     # Crée un qdisc HTB racine avec l'identifiant 1:
     # - default 999: les paquets non classifiés vont dans la classe 999
     tc qdisc add dev "$IF" root handle 1: htb default 999
```

```
      # Crée une classe par défaut avec un débit très limité (128kbit)
      # - Cette classe gère le trafic qui ne correspond à aucune règle
      tc class add dev "$IF" parent 1: classid 1:999 htb rate 128kbit ceil 128kbit
      # Ajoute fq_codel à cette classe pour gérer la latence sous charge
      tc qdisc add dev "$IF" parent 1:999 fq_codel
    fi
}

# Fonction qui crée des classes de limitation et des filtres pour chaque IP du LAN
add_classes_and_filters() {
    local IF=$1 MODE=$2 FIELD
    # Détermine si on filtre par source ou destination en fonction du mode
    case "$MODE" in
      src) FIELD="src";;  # Pour limiter le trafic sortant (upload)
      dst) FIELD="dst";;  # Pour limiter le trafic entrant (download)
      *) echo "Mode invalide"; exit 1;;
    esac

    # Assure qu'un qdisc racine existe sur l'interface
    maybe_add_root "$IF"

    # Parcourt toutes les adresses IP potentielles du LAN
    for ((i=HOST_START;i<=HOST_END;i++)); do
      # Ignore l'IP du pare-feu (ne pas la limiter)
      [[ $i -eq $FIREWALL_SUFFIX ]] && continue

      # Construit l'adresse IP complète et l'identifiant de classe
      IP="${LAN_BASE}.${i}"
      CLASSID="1:1${i}"  # Format: 1:1X où X est le dernier octet de l'IP

      # Si la classe n'existe pas déjà, la créer
      if ! tc class show dev "$IF" | grep -q "class htb ${CLASSID}"; then
        log "Classe $CLASSID pour $IP ($FIELD) sur $IF"
        # Crée une classe HTB avec:
        # - rate: débit garanti (4000kbit)
        # - ceil: débit maximum (même valeur = pas de partage de bande passante
excédentaire)
        # - burst: tolérance pour les pics courts
        tc class add dev "$IF" parent 1: classid "$CLASSID" htb rate "$RATE" ceil
"$CEIL" burst "$BURST"
        # Ajoute fq_codel à la classe pour une meilleure gestion de la latence
        tc qdisc add dev "$IF" parent "$CLASSID" fq_codel
      fi

      # Si le filtre n'existe pas déjà, le créer
      if ! tc filter show dev "$IF" parent 1: | grep -q "flowid $CLASSID.* $FIELD
$IP"; then
        log "Filtre $FIELD=$IP -> $CLASSID sur $IF"
        # Crée un filtre u32 qui:
        # - Correspond aux paquets ayant l'IP spécifiée comme source ou destination
        # - Les dirige vers la classe correspondante
        tc filter add dev "$IF" parent 1: protocol ip prio 1 u32 \
          match ip $FIELD $IP/32 flowid "$CLASSID"
      fi
```

```
  done
}

# Application des limites pour les trois cas de figure:
add_classes_and_filters "$LAN_IF" dst   # 1. Download vers LAN (limite par IP de
destination)
add_classes_and_filters "$WAN_IF" src   # 2. Upload vers WAN (limite par IP
source)
add_classes_and_filters "$DMZ_IF" src   # 3. Upload vers DMZ (limite par IP
source)
```

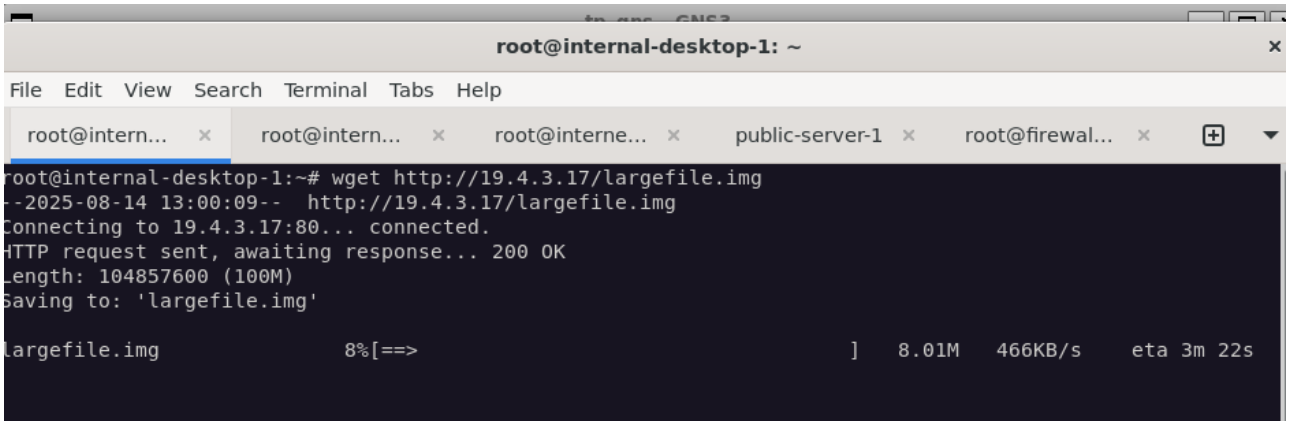
5. Tests fonctionnels et vérification

Pour vérifier que les limites de bande passante sont bien appliquées, on peut utiliser plusieurs outils:

```
# Sur le serveur WAN (`internet`)
dd if=/dev/zero of=/var/www/html/largefile.img bs=1M count=100

# Dans `internal-desktop`: tester la limitation de bande passante
wget http://19.4.3.17/largefile.img
```

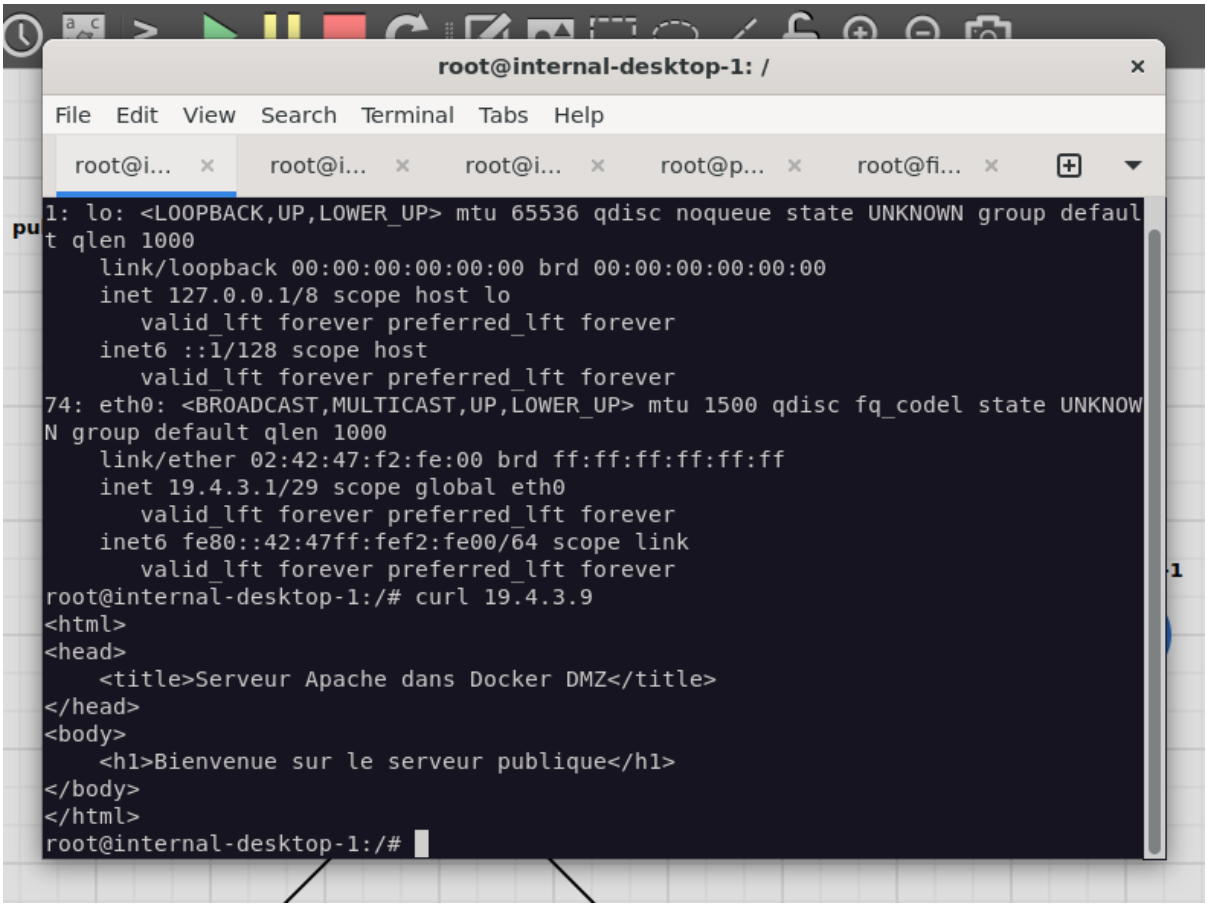
Résultats:

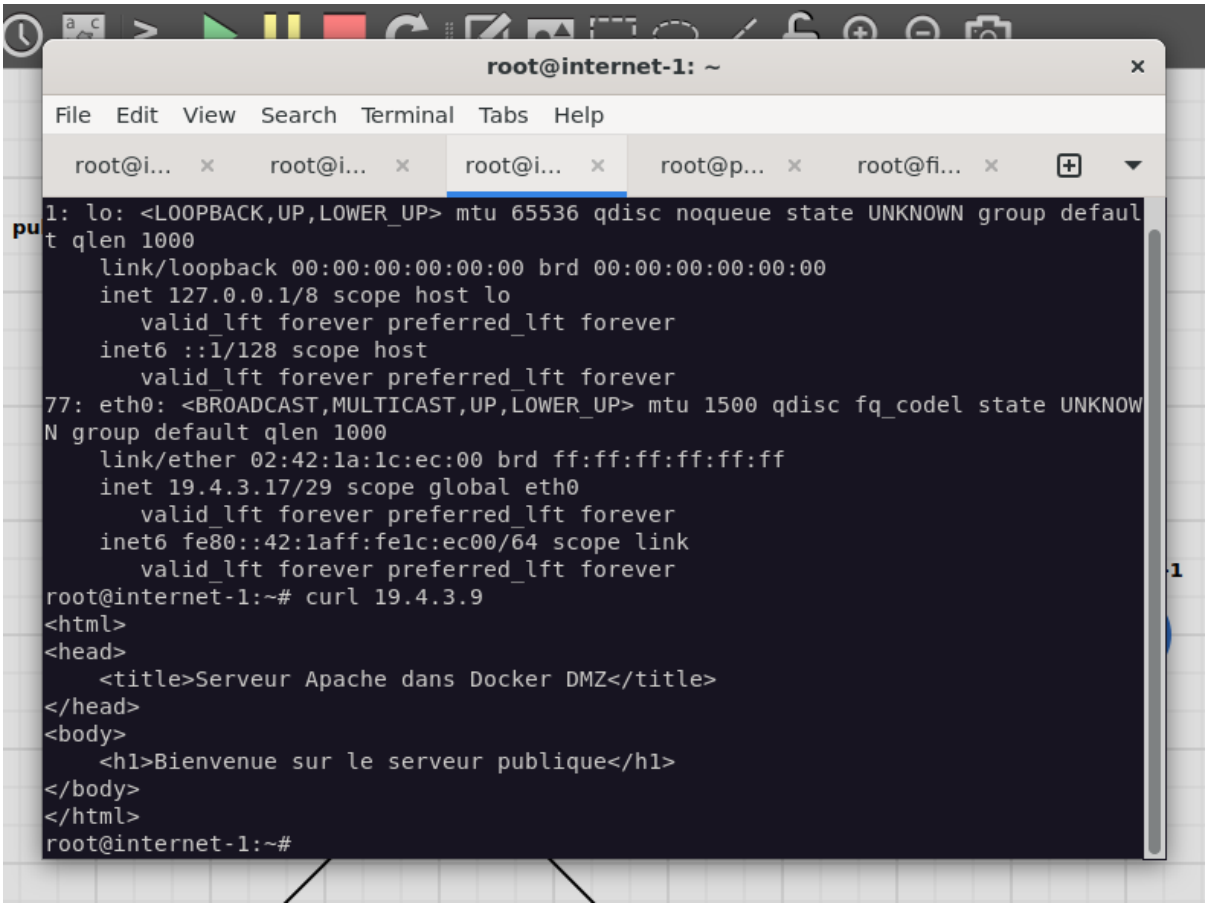
- Les vitesses de transfert ne devraient pas dépasser ~500 Ko/s par machine
- Les transferts sont régulés sans pertes de paquets excessives
- 

3.2. DMZ

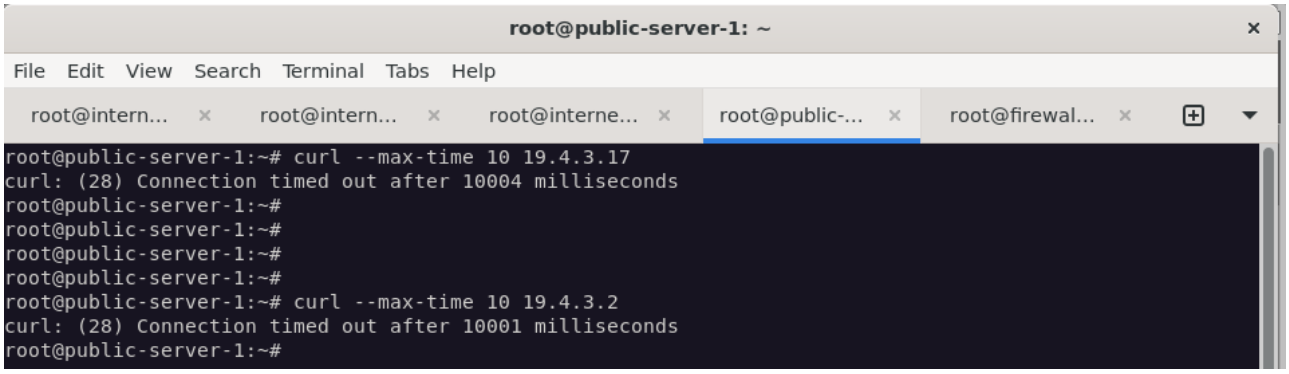
Accessible en HTTP/HTTPS depuis WAN et LAN et **Ne peut initier aucune connexion sortante** sont déjaà realisés avec la configuration du pare-feu

```
iptables -A FORWARD -p tcp -s $LAN,$WAN -d $DMZ -m multiport --dports
$HTTP,$HTTPS,$SSH -m conntrack --ctstate NEW,ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -s $DMZ -d $LAN,$WAN -m conntrack --ctstate
ESTABLISHED,RELATED -j ACCEPT
```

- LAN -> DMZ
  - 

- WAN -> DMZ
  - 

- DMZ -> LAN et LAN (initiation d'une connexion sortante vers internet puis vers le server interne)



3.3. Surveillance IDS avec Snort

Detecter et journaliser les tentatives d'attaque et de scan

Justification : L'implémentation d'un système de détection d'intrusion (IDS) comme Snort constitue une couche de défense essentielle pour notre DMZ. Contrairement au pare-feu qui se concentre sur le filtrage du trafic selon des règles préétablies, l'IDS analyse en profondeur le contenu des paquets pour identifier des signatures d'attaque connues et des comportements suspects. Ce système nous permet de détecter rapidement les tentatives de reconnaissance (scans de ports), les attaques d'injection (SQL, XSS) et d'autres vecteurs d'attaque, même si le trafic utilise des ports autorisés. La journalisation des événements fournit une traçabilité précieuse pour l'analyse forensique en cas d'incident et permet d'affiner continuellement notre politique de sécurité. Le placement de Snort directement dans la DMZ est stratégique car cette zone est particulièrement exposée aux menaces externes, étant accessible depuis Internet.

1. Configuration de base de Snort

On commence par faire un backup du fichier de configuration principal de snort

```
cp /etc/snort/snort.conf /etc/snort/snort.conf.bak
```

Nous allons configurer Snort pour qu'il détecte et journalise les tentatives d'attaque et de scan sur notre serveur DMZ (19.4.3.9).

2. Modification des variables réseau

D'abord, nous modifions les variables de réseau dans snort.conf pour cibler spécifiquement notre serveur DMZ:

```
# Dans /etc/snort/snort.conf
ipvar HOME_NET 19.4.3.9
ipvar EXTERNAL_NET !$HOME_NET
```

3. Création de règles de détection personnalisées

Nous créons ensuite un fichier de règles local pour détecter différents types d'attaques:

```
# Créer le fichier local.rules
cat > /etc/snort/rules/local.rules << 'EOF'

# ==== SCANS DE PORT ====
alert tcp any any -> $HOME_NET any (msg:"[SCAN] TCP SYN scan détecté"; flags:S,12;
threshold:type threshold, track by_src, count 5, seconds 60; classtype:attempted-
recon; sid:2000001; rev:1;)

alert tcp any any -> $HOME_NET any (msg:"[SCAN] TCP Connect scan détecté";
flags:S; threshold:type threshold, track by_src, count 10, seconds 60;
classtype:attempted-recon; sid:2000002; rev:1;)

alert udp any any -> $HOME_NET any (msg:"[SCAN] UDP scan détecté"; threshold:type
threshold, track by_src, count 15, seconds 60; classtype:attempted-recon;
sid:2000003; rev:1;)

# ==== ATTAQUES DoS ====
alert icmp any any -> $HOME_NET any (msg:"[DoS] Attaque ICMP flood détectée";
threshold:type threshold, track by_src, count 30, seconds 10; classtype:attempted-
dos; sid:2000010; rev:1;)

alert tcp any any -> $HOME_NET any (msg:"[DoS] Attaque SYN flood détectée";
flags:S,12; flow:stateless; threshold:type threshold, track by_src, count 50,
seconds 10; classtype:attempted-dos; sid:2000011; rev:1;)

# ==== ATTAQUES WEB ====
alert tcp any any -> $HOME_NET 80 (msg:"[WEB] Possible injection SQL détectée";
content:"UNION SELECT"; nocase; classtype:web-application-attack; sid:2000020;
rev:1;)

alert tcp any any -> $HOME_NET 80 (msg:"[WEB] Possible attaque XSS détectée";
content:"<script>"; nocase; classtype:web-application-attack; sid:2000022; rev:1;)
EOF
```

4. Configuration de la journalisation

Pour assurer un suivi complet des attaques détectées, nous configurons plusieurs formats de journalisation dans snort.conf:

```
# Dans /etc/snort/snort.conf
output log_tcpdump: /var/log/snort/tcpdump.log
output alert_fast : /var/log/snort/alert.fast
```

5. Démarrage de Snort avec la nouvelle configuration

Enfin, nous démarrons Snort avec la configuration mise à jour:

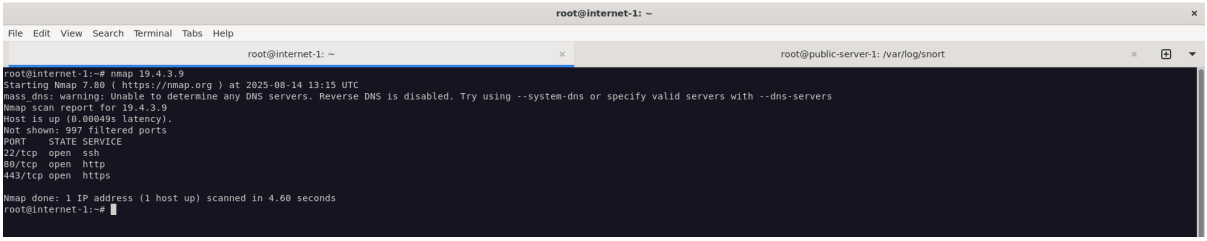
```
# Créer le répertoire pour les logs
mkdir -p /var/log/snort/

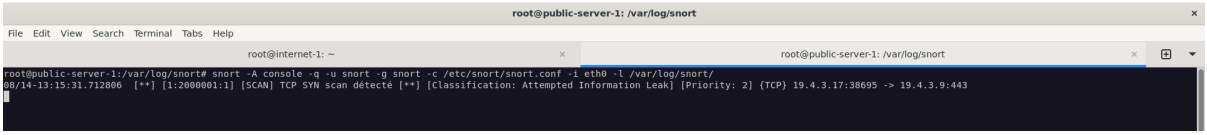
chown -R snort:snort /var/log/snort/

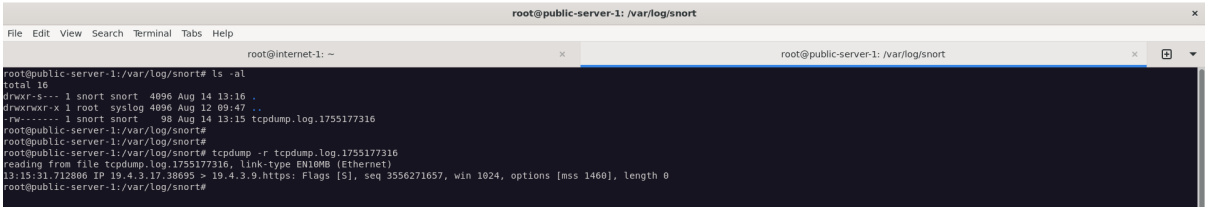
# Verifier si les syntax sont correctes
snort -T -c /etc/snort/snort.conf

service snort restart

# Demarrer snort
snort -D -q -u snort -g snort -c /etc/snort/snort.conf -i eth0 -l /var/log/snort/
```

- nmap depuis internet vers la DMZ
  - 

- detection de nmap par snort dans la DMZ
  - 

- log snort pour les scans de nmap
  - 

Résultats des tests

Cette section présente une synthèse des différents tests effectués pour valider le bon fonctionnement de notre architecture DMZ et des mécanismes de sécurité mis en place.

Connectivité et isolation des réseaux

Test	Résultat	Acteurs impliqués	Commande utilisée	Observation
LAN → DMZ (HTTP/HTTPS)	Réussi	Depuis `internal-desktop` vers `public-server`	`curl http://19.4.3.9` `curl -k https://19.4.3.9`	Le trafic est correctement autorisé depuis le LAN vers les services web de la DMZ

Test	Résultat	Acteurs impliqués	Commande utilisée	Observation
WAN → DMZ (HTTP/HTTPS)	Réussi	Depuis `internet` vers `public-server`	`curl http://19.4.3.9` `curl -k https://19.4.3.9`	Le trafic est correctement autorisé depuis le WAN vers les services web de la DMZ
LAN → WAN	Réussi	Depuis `internal-desktop` vers `internet`	`ping 19.4.3.17`	Le trafic sortant du LAN est correctement masqué par NAT
WAN → LAN	Bloqué	Depuis `internet` vers `internal-server`	`curl http://19.4.3.2`	Aucun accès direct n'est possible depuis le WAN vers le LAN, comme prévu
DMZ → LAN	Bloqué	Depuis `public-server` vers `internal-server`	`ping 19.4.3.2`	La DMZ ne peut pas initier de connexion vers le LAN, assurant une bonne isolation

Filtrage de contenu

Test	Résultat	Acteurs impliqués	Commande utilisée	Observation
Accès aux sites pornographiques	Bloqué	Depuis `internal-desktop` via `firewall` (SquidGuard)	`curl http://pornhub.com`	Le filtrage SquidGuard bloque efficacement l'accès aux sites pornographiques
Accès aux réseaux sociaux (heures de travail)	Bloqué	Configuration sur `firewall` puis test depuis `internal-desktop`	`date -s "09:30:00"` `curl http://facebook.com`	L'accès est bloqué pendant les heures de travail définies
Accès aux réseaux sociaux (hors heures)	Autorisé	Configuration sur `firewall` puis test depuis `internal-desktop`	`date -s "13:30:00"` `curl http://facebook.com`	L'accès est correctement autorisé en dehors des heures de travail

Qualité de service et limitation de bande passante

Test	Résultat	Acteurs impliqués	Commandes utilisées	Observation

Test	Résultat	Acteurs impliqués	Commandes utilisées	Observation

Téléchargement de fichier volumineux	Limité	Depuis `internal-desktop` vers `internet` à travers les règles QoS du `firewall`	Sur le serveur WAN (`internet`) :
			`dd if=/dev/zero of=/var/www/html/largefile.img bs=1M count=100`
			
			Sur le client LAN (`internal-desktop`) :
			`wget http://19.4.3.17/largefile.img`
				La vitesse est effectivement plafonnée à environ 500 KB/s

Détection d'intrusion

Test	Résultat	Acteurs impliqués	Commandes utilisées	Observation

Scan de ports depuis le WAN	Détecté	Attaque depuis `internet`, détection par `snort` sur `public-server`	Sur `internet` :
			`nmap -sS 19.4.3.9`
			
			Sur `public-server` pour vérifier les logs :
			`cat /var/log/snort/alert.fast`
				Snort détecte et journalise correctement les tentatives de scan de ports

Simulation d'attaques web	Détecté	Attaque depuis `internet`, détection par `snort` sur `public-server`	Sur `internet` :
			`curl "http://19.4.3.9/?id=1 UNION SELECT username,password FROM users"`
			`curl "http://19.4.3.9/?data=<script>alert('XSS')</script>"`
				Les requêtes suspectes contenant des motifs d'attaque sont identifiées

Analyse et conclusion

Notre mise en œuvre d'une architecture DMZ via GNS3 et Docker a permis de créer un environnement réseau sécurisé qui répond aux objectifs initiaux. Cette section présente une analyse des résultats obtenus et les conclusions tirées de ce travail pratique.

Réalisation des objectifs

1. **Isolation des réseaux** : La DMZ a été correctement isolée du LAN interne, permettant d'héberger des services accessibles depuis l'extérieur sans compromettre la sécurité du réseau interne.
2. **Filtrage du trafic** : Le pare-feu a été configuré pour filtrer efficacement le trafic entre les différentes zones (WAN, DMZ, LAN) selon les règles établies. Les tests ont confirmé que seuls les services autorisés (HTTP, HTTPS, SSH) sont accessibles depuis le LAN et le WAN vers la DMZ.

3. **Protection contre les attaques** : L'intégration de Snort a démontré la capacité à détecter et journaliser diverses tentatives d'attaque et de scan, comme montré lors des tests avec nmap.

4. **Contrôle d'accès au contenu** : Le système de filtrage avec Squid et SquidGuard a permis de bloquer efficacement les réseaux sociaux pendant les heures de travail et les sites pornographiques .

5. **Qualité de service** : La limitation de bande passante à 500 KB/s pour chaque machine du LAN assure une distribution équitable des ressources réseau.

Difficultés rencontrées

1. **Configuration de SquidGuard** : La mise en place du filtrage de contenu a nécessité une attention particulière aux permissions des fichiers et à la syntaxe du fichier de configuration.

2. **Intégration de Snort** : La configuration des règles de détection pour cibler spécifiquement notre environnement a requis plusieurs ajustements.

3. **Absence d'accès Internet** : Dans notre environnement de simulation, nous n'avons pas pu avoir un accès réel à Internet, ce qui a limité certains tests notamment pour le filtrage de contenu et la résolution DNS. Nous avons dû simuler ces aspects en utilisant des entrées DNS locales et des contenus hébergés dans notre propre infrastructure.

Conclusion

Ce travail pratique nous a permis d'appliquer concrètement les concepts théoriques de sécurité réseau à travers la mise en place d'une DMZ fonctionnelle. La combinaison de GNS3 et Docker s'est révélée être un choix pertinent, offrant la flexibilité nécessaire pour simuler une infrastructure réseau complexe.

Les compétences développées lors de ce projet incluent la configuration avancée de pare-feu, la mise en œuvre de systèmes de filtrage de contenu, et l'utilisation d'outils de détection d'intrusion.

En conclusion, malgré quelques difficultés techniques, l'objectif principal de créer une architecture DMZ sécurisée a été atteint avec succès. Les tests effectués ont validé le bon fonctionnement de l'ensemble des mécanismes de sécurité mis en place, démontrant ainsi la robustesse de notre solution.